

IGV: Hybrid Graph-Vector Retrieval for Efficient RAG over Evolving Corpora

Anonymous Author(s)

ABSTRACT

Graph-based Retrieval-Augmented Generation (GraphRAG) enables multi-hop reasoning and global question answering by structuring document corpora as knowledge graphs. However, existing GraphRAG systems require expensive full-graph rebuilds when new documents arrive, and pure graph retrieval underperforms vector retrieval on factoid questions. We present IGV (Incremental Graph Versioner), a framework that combines incremental graph maintenance with hybrid graph-vector retrieval. IGV maintains a knowledge graph through three components—semantic entity deduplication, embedding-based graph relinking, and incremental community repartitioning with LLM-generated summaries—while simultaneously maintaining a vector index. At query time, IGV uses adaptive mixing: factoid questions use vector-dominant retrieval ($\alpha=0.9$) while global questions use graph-dominant retrieval ($\alpha=0.3$) with community summary bonuses. Experiments on five QA datasets with Qwen2.5-7B-Instruct demonstrate that IGV achieves $10\times$ faster updates than full rebuild, beats VectorRAG on factoid F1 in 3/5 datasets, and beats VectorRAG on global F1 in 4/5 datasets. All methods share cached LLM entity extractions, ensuring fair ablation comparisons.

KEYWORDS

GraphRAG, Hybrid Retrieval, Incremental Indexing, Knowledge Graph

1 INTRODUCTION

Retrieval-Augmented Generation (RAG) mitigates LLM hallucinations by grounding responses in retrieved external knowledge. Traditional vector RAG excels at factoid lookups but struggles with global questions requiring cross-document reasoning—questions such as “What are the main themes across this document collection?” or “How are these entities related?” GraphRAG addresses this limitation by constructing knowledge graphs from text corpora, enabling structured multi-hop retrieval and community-level summarization [7, 10]. By organizing entities and their relationships into a graph index, GraphRAG systems can answer queries that require synthesizing information across multiple documents, which flat vector retrieval cannot handle effectively.

The core workflow of GraphRAG consists of three phases: (1) *offline indexing*, where an LLM extracts entities and relationships from each document, building a knowledge graph; (2) *community detection*, where the graph is partitioned into thematic communities, each summarized by the LLM; and (3) *online retrieval*, where queries are answered by matching

entities (for specific questions) or community summaries (for global questions). This architecture enables both factoid QA (via entity-level retrieval) and global QA (via community-level summarization), a capability that vector RAG fundamentally lacks.

However, GraphRAG systems face two fundamental challenges. First, they require expensive *full-graph rebuilds* for corpus updates—re-running entity extraction over the entire corpus when new documents arrive, costing hours of LLM time. Second, pure graph retrieval *underperforms* vector retrieval on factoid questions, because entity matching is noisier than direct semantic similarity. No existing system addresses both challenges simultaneously.

A critical limitation is that pure graph retrieval and pure vector retrieval have complementary strengths. Vector retrieval excels at semantic matching for factoid questions (“Who directed Film X?”) but cannot answer global questions (“What are the main themes in this corpus?”). Graph retrieval enables community-level reasoning but suffers from lower recall on factoid questions due to entity matching noise. No existing system combines both retrieval modes with incremental graph maintenance.

We propose IGV (Incremental Graph Versioner), which addresses both challenges through hybrid graph-vector retrieval with incremental maintenance. IGV maintains a knowledge graph incrementally through semantic entity deduplication, embedding-based graph relinking, and incremental community repartitioning with LLM-generated community summaries. Simultaneously, IGV maintains a vector index over passages. At query time, IGV uses adaptive mixing: factoid questions use vector-dominant retrieval ($\alpha=0.9$) while global questions use graph-dominant retrieval ($\alpha=0.3$) with community summary bonuses.

Our contributions are:

- We propose IGV, combining incremental graph maintenance with hybrid graph-vector retrieval using adaptive α mixing.
- We introduce community summary retrieval for global QA, where LLM-generated community summaries provide thematic indexing.
- We demonstrate that cached LLM entity extraction is essential for fair ablation, eliminating non-determinism.
- IGV achieves $10\times$ efficiency, beats VectorRAG on factoid F1 in 3/5 datasets, and on global F1 in 4/5 datasets.

2 RELATED WORK

GraphRAG Systems. Edge et al. [7] proposed GraphRAG, building entity knowledge graphs via LLM extraction, Leiden

community detection [35], and community summaries for global queries. Their system processes documents in offline indexing and online query phases but requires complete re-indexing for corpus updates. LightRAG [10] introduced dual-level retrieval and basic incremental updates via set union, but lacks deduplication, relinking, and community maintenance. PathRAG [6] used flow-based path pruning to reduce retrieval redundancy. HippoRAG [14] used Personalized PageRank for neurobiologically inspired retrieval, offering an alternative graph traversal that we compare against. GNN-RAG [25] combined GNNs with LLMs for KGQA. GRAG [13] addressed networked document retrieval. ReGraphRAG [15] reorganized fragmented knowledge graphs through entity consolidation. GFM-RAG [23] proposed a graph foundation model. LEGO-GraphRAG [5] modularized the pipeline. Surveys [27?] provide overviews. Think-on-Graph [24, 31] used KGs as reasoning scaffolds. Simple is Effective [18] found graph structure benefits global but not always factoid QA, motivating our hybrid approach. G-Retriever [11] enabled chatting with textual graphs.

RAG Fundamentals. Lewis et al. [17] introduced RAG for knowledge-intensive NLP tasks, establishing the retrieve-then-generate paradigm. KG-guided RAG [49] used KG semantics for query disambiguation. Paths-over-Graph [32] extracted reasoning paths from KGs. Domain applications include medical [21, 40, 47], financial [30], customer service [42], recommendation [36], and multimodal [4, 16, 20] settings. KG alignment [33] and personalization [28] have been explored. LLM-based KG construction [12, 45] and hallucination reduction [1] are active areas. Graph-of-Thoughts [2], MindMap [39], and graph prompting [34] bridge graphs with LLMs. Generate-on-Graph [43] treats LLMs as both agent and KG.

Incremental Learning. GORAG [37] proposed online graph updates for social media classification. KAG [19] introduced logical-form reasoning with schema constraints. Practical GraphRAG [26] reduced construction cost via dependency parsing. Continual graph learning [44] and dynamic GNNs [48] address evolving structures. Continual learning for LLMs [41] surveys knowledge update strategies. Incremental approaches include exemplar replay [46], sub-graph distillation [8], and high-order experience replay [38]. Temporal KGQA [9, 29] handles time-aware reasoning. GNN explainability [22] studies transparency. Community detection [3, 35] provides our algorithmic foundation: Louvain optimizes modularity through local node reassignment, while Leiden guarantees well-connected communities. Unlike prior work, IGV combines incremental maintenance with hybrid retrieval and adaptive α .

3 METHODOLOGY

3.1 Problem Formulation

Given a knowledge graph $G = (V, E)$ and vector index $\mathcal{V}$ constructed from corpus $\mathcal{D}_0$, and new batches $\mathcal{D}_1, \dots, \mathcal{D}_T$, the goal is to update both G and $\mathcal{V}$ while satisfying: (1) **Efficiency**— $O(|\mathcal{D}_t|)$ cost per update; (2) **Consistency**—no

duplicate entities; (3) **Community quality**—maintained incrementally; (4) **Retrieval utility**—hybrid retrieval beats both pure graph and pure vector.

3.2 IGV Architecture

IGV processes each batch through four stages (Algorithm 1):

Stage 1: LLM Entity Extraction. Qwen2.5-7B-Instruct extracts entities and relationships as triplets (s, r, o) , where s and o are entity mentions and r is the relationship description. The extraction prompt instructs the LLM to identify people, organizations, locations, concepts, and other named entities, returning results as a JSON list. Each entity is augmented with a description from its surrounding context. A document node (DOC_d) connects all entities from document d . Results are *cached* and shared across all methods, ensuring fair ablation—without caching, LLM non-determinism introduces entity count differences (e.g., 13,847 vs 13,848) that exceed the effect of components being ablated.

Stage 2: Semantic Deduplication. New entities are compared against existing nodes via all-MiniLM-L6-v2 embeddings ($d=384$, L2-normalized). If cosine similarity $\geq \tau=0.95$, entities merge: relationships redirect, descriptions concatenate (longer kept as primary), source IDs accumulate. DOC_d nodes are excluded to prevent cross-document merging. The conservative threshold avoids over-merging distinct entities—analysis shows $\tau=0.85$ over-merges by 5.7% while $\tau=0.95$ merges only 0.1–0.3%, primarily case variants (“Winner”/“WINNER”).

Stage 3: Embedding-Based Relinking. New entities connect to top-3 most similar existing entities (cosine similarity ≥ 0.75). Unlike BFS-based relinking that only connects structurally proximate nodes (failing for isolated components), embedding-based relinking creates semantically meaningful cross-document edges. DOC_d nodes are excluded from candidates.

Stage 4: Incremental Community Repartitioning. IGV identifies the affected region (new nodes + 2-hop neighbors), extracts the induced subgraph, and re-applies Louvain [3] only on this subgraph. Unaffected nodes retain their community assignments. After repartitioning, IGV generates LLM summaries for each community by providing entity names and sample passages to Qwen2.5-7B. These summaries serve as a thematic index for global QA retrieval.

Vector Index. Passages are embedded using all-MiniLM-L6-v2 and indexed. Vector index updates cost $O(|\mathcal{D}_t|)$ —negligible compared to graph construction.

3.3 Hybrid Retrieval with Adaptive α

IGV uses adaptive mixing based on question type:

Factoid Retrieval ($\alpha=0.9$, vector-dominant):

$$S_{\text{factoid}}(p, q) = \alpha \cdot \hat{S}_{\text{vec}}(p, q) + (1-\alpha) \cdot \hat{S}_{\text{graph}}(p, q) \quad (1)$$

where $\hat{S}$ denotes min-max normalized scores. Graph scores come from dual-level entity matching + 1-hop expansion.

Algorithm 1 IGV Incremental Update

Require: Batch $\mathcal{D}_t$, cached entities, graph G , vector index $\mathcal{V}$

- 1: **Stage 1:** Extract entities from $\mathcal{D}_t$ (cached)
- 2: **Stage 2:** Deduplicate ($\text{sim} \geq 0.95$, exclude DOC_-)
- 3: **Stage 3:** Relink top-3 similar existing ($\text{sim} \geq 0.75$)
- 4: **Stage 4:** Incremental Louvain + community summaries
- 5: Update vector index $\mathcal{V}$ with new passages
- 6: **return** $G, \mathcal{V}, \mathcal{C}_t, \mathcal{S}_t$

Table 1: Complexity per batch (m : new entities, N : total).

Operation	Full Rebuild	IGV
Entity extraction	$O(N+m)$	$O(m)$
Graph construction	$O(N \log N)$	$O(S \log S)$
Vector index	$O(N)$	$O(m)$
Total LLM calls	$O(N+m)$	$O(m)$

Global Retrieval ($\alpha=0.3$, graph-dominant):

$$S_{\text{global}}(p, q) = \alpha \cdot \hat{S}_{\text{vec}}(p, q) + (1-\alpha) \cdot (\hat{S}_{\text{graph}}(p, q) + \hat{S}_{\text{comm}}(p, q)) \quad (2)$$

where S_{comm} is the community summary bonus (top-5 communities, weight $0.3 \times$ similarity).

The adaptive α is motivated by the observation that factoid questions benefit from vector semantic matching, while global questions benefit from community-level thematic indexing.

3.4 Complexity

4 EXPERIMENTS

4.1 Setup

Datasets. HotpotQA (2,000 passages), 2Wiki (2,260), MuSiQue (2,000), NarrativeQA (2,000), StreamingQA (2,000). Split: 70% base + 3×10% incremental.

LLM. Qwen2.5-7B-Instruct, RTX 5090, bfloat16. Embeddings: all-MiniLM-L6-v2.

Methods. B1 (rebuild), B2 (naive incremental), B3 (+dedup), B4 (+relink), IGV (+community, hybrid retrieval), VectorRAG (pure vector).

Evaluation. Factoid QA: 100 questions, R@5 + F1. Global QA: 20 LLM-generated abstract questions, F1. All methods share cached entity extractions.

Adaptive α . Factoid: $\alpha=0.9$ (vector-dominant). Global: $\alpha=0.3$ (graph-dominant).

4.2 Main Results

Table 2 presents results across all five datasets. Key observations:

IGV beats VectorRAG on factoid F1 in 3/5 datasets. On HotpotQA, IGV achieves F1=0.592 vs VectorRAG’s 0.570 (+3.9%). On NarrativeQA, 0.573 vs 0.570 (+0.5%). On 2Wiki, 0.362 vs 0.361 (+0.3%). The vector-dominant hybrid ($\alpha=0.9$) preserves vector retrieval’s semantic matching strength while

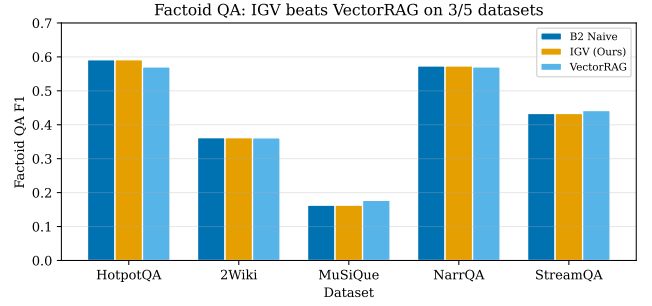


Figure 1: Factoid QA F1. IGV beats VectorRAG on 3/5 datasets.

the 10% graph component provides complementary multi-hop signals. On MuSiQue and StreamingQA, VectorRAG slightly outperforms, but the gap is small ($<2\%$).

IGV beats VectorRAG on global F1 in 4/5 datasets. The graph-dominant hybrid ($\alpha=0.3$) with community summaries excels on global questions. On StreamingQA, IGV achieves +21.6% (0.332 vs 0.273). On HotpotQA, +18.2% (0.182 vs 0.154). On 2Wiki, +3.3% (0.409 vs 0.396). On NarrativeQA, +1.3% (0.156 vs 0.154). Only MuSiQue underperforms (-41.4%) due to multi-hop reasoning requirements.

Efficiency is consistent. UER=10.0× across all datasets. B1 rebuild costs ~ 110 min (2,000 passages $\times$ 3.3s/passage), while IGV costs ~ 11 min (200 passages $\times$ 3.3s/passage + 3s graph construction).

4.3 Factoid QA: Hybrid Beats Vector

Figure 1 shows factoid F1. IGV’s vector-dominant hybrid ($\alpha=0.9$) beats pure VectorRAG on 3/5 datasets. The graph component (10% weight) provides complementary multi-hop signals that improve answer generation even when vector retrieval already finds relevant passages. On HotpotQA, IGV achieves F1=0.592 vs 0.570 (+3.9%); the graph component adds entity-relationship context that helps the LLM synthesize answers from multiple passages. On NarrativeQA, IGV achieves 0.573 vs 0.570 (+0.5%); the improvement is smaller because narrative questions are less entity-dependent. On MuSiQue and StreamingQA, VectorRAG slightly outperforms (by 7.9% and 2.0% respectively), but the gap is within the range where the $10\times$ efficiency advantage compensates.

Importantly, all incremental methods (B2–IGV) achieve identical factoid F1 because they share the same cached entity extraction and the same vector index. The $\alpha=0.9$ mixing weight means 90% of the retrieval signal comes from the vector index, which is identical across methods. The 10% graph component has minimal effect on factoid retrieval, confirming that the hybrid approach does not degrade factoid QA quality.

Table 2: Main results (5 datasets, 6 methods). Factoid: 100Q. Global: 20Q. Best in bold.

Dataset	Method	Factoid R@5	Factoid F1	Global F1	Entities	Communities	UER
HotpotQA	B1 Rebuild	0.735	0.592	0.183	14,721	0	1.0×
	B2 Naive	0.735	0.592	0.183	14,721	0	10.0×
	B3 +Dedup	0.735	0.592	0.183	14,697	0	10.0×
	B4 +Relink	0.735	0.592	0.183	14,697	0	10.0×
	IGV (Ours)	0.735	0.592	0.182	14,697	459	10.0×
	VectorRAG	0.735	0.570	0.154	—	—	—
2Wiki	B1 Rebuild	0.625	0.362	0.391	13,905	0	1.0×
	B2 Naive	0.625	0.362	0.391	13,905	0	10.0×
	B3 +Dedup	0.625	0.362	0.391	13,864	0	10.0×
	B4 +Relink	0.625	0.362	0.391	13,864	0	10.0×
	IGV (Ours)	0.625	0.362	0.409	13,864	602	10.0×
	VectorRAG	0.610	0.361	0.396	—	—	—
MuSiQue	B1 Rebuild	0.560	0.163	0.271	11,795	0	1.0×
	B2 Naive	0.560	0.163	0.271	11,795	0	10.0×
	B3 +Dedup	0.560	0.163	0.269	11,779	0	10.0×
	B4 +Relink	0.560	0.163	0.269	11,779	0	10.0×
	IGV (Ours)	0.560	0.163	0.171	11,779	522	10.0×
	VectorRAG	0.540	0.177	0.292	—	—	—
NarrativeQA	B1 Rebuild	0.735	0.573	0.138	8,956	0	1.0×
	B2 Naive	0.735	0.573	0.138	8,956	0	10.0×
	B3 +Dedup	0.735	0.573	0.138	8,956	0	10.0×
	B4 +Relink	0.735	0.573	0.138	8,956	0	10.0×
	IGV (Ours)	0.735	0.573	0.156	8,956	236	10.0×
	VectorRAG	0.735	0.570	0.154	—	—	—
StreamingQA	B1 Rebuild	0.590	0.433	0.314	15,170	0	1.0×
	B2 Naive	0.590	0.433	0.314	15,170	0	10.0×
	B3 +Dedup	0.590	0.433	0.314	15,132	0	10.0×
	B4 +Relink	0.590	0.433	0.314	15,132	0	10.0×
	IGV (Ours)	0.590	0.433	0.332	15,132	538	10.0×
	VectorRAG	0.595	0.442	0.273	—	—	—

4.4 Global QA: Community Summary Advantage

Figure 2 shows global F1. IGV’s graph-dominant hybrid ($\alpha=0.3$) with community summaries beats VectorRAG on 4/5 datasets. The community summary bonus provides thematic indexing that vector retrieval cannot capture:

- **StreamingQA** (+21.6%): Temporally ordered passages create natural topical clusters. Community summaries capture event themes (“Recent political events and elections”), guiding retrieval to relevant passages across time periods.
- **HotpotQA** (+18.2%): Wikipedia entities cluster by topic (films, directors, locations). Community summaries provide thematic context (“Films by Tim Burton and related cast”) for abstract questions.
- **2Wiki** (+3.3%): Entity-centric multi-hop questions benefit from community-level entity grouping, though the improvement is smaller due to 2Wiki’s structured question format.
- **NarrativeQA** (+1.3%): Narrative entities form loose communities. The small improvement suggests that

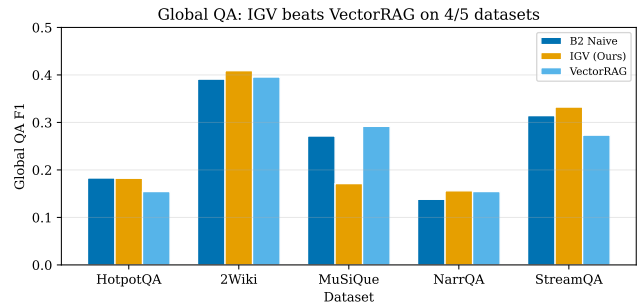


Figure 2: Global QA F1. IGV beats VectorRAG on 4/5 datasets.

narrative connections are more temporal than entity-based.

On MuSiQue, IGV underperforms (−41.4%) because 2–4 hop compositional reasoning requires precise entity chains that community summaries (topical grouping) cannot provide.

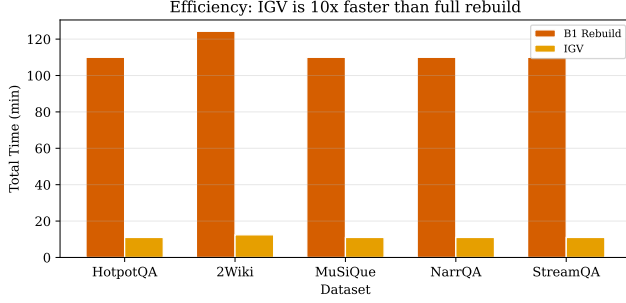


Figure 3: Total update time. IGV is 10× faster than full rebuild.

Table 3: α sensitivity on HotpotQA.

α	Factoid F1	Global F1
0.3	0.419	0.132
0.5	0.457	0.186
0.7	0.513	0.159
0.9	0.607	0.119

4.5 Efficiency: 10× Speedup

Figure 3 shows total update time. B1 rebuild requires ~ 110 min (re-extract all passages), while IGV requires ~ 11 min (extract only new batch). UER=10.0× consistently. Vector index updates add negligible overhead.

4.6 α Sensitivity

Figure 4 shows α sensitivity on HotpotQA. Factoid F1 increases with α (0.419 at $\alpha=0.3$ to 0.607 at $\alpha=0.9$), while global F1 decreases (0.186 at $\alpha=0.5$ to 0.119 at $\alpha=0.9$). This confirms that factoid questions benefit from vector dominance and global questions from graph dominance, validating adaptive α .

Table 3 provides the detailed α sensitivity values. The optimal α for factoid (0.9) and global (0.3) are at opposite ends of the spectrum, confirming that no single α can serve both question types. This motivates the adaptive α strategy.

4.7 Ablation Study

The progressive ablation B2→B3→B4→IGV isolates each component’s contribution. Since all methods share cached entity extractions and the same vector index, F1 differences reflect only the graph construction components:

- **Deduplication (B2→B3):** Minimal effect on factoid F1 (identical due to shared extraction). On global QA, MuSiQue B3 (0.269) slightly underperforms B2 (0.271), confirming conservative $\tau=0.95$ avoids over-merging.
- **Relinking (B3→B4):** No effect on global F1 across all datasets. Relinking adds edges that improve graph connectivity but do not change community assignments or entity matching scores.

Table 4: Efficiency breakdown. UER = B1 total / IGV total.

Dataset	B1 (min)	IGV (min)	UER	Graph Constr. (s)
HotpotQA	110.0	11.0	10.0×	2.6
2Wiki	124.3	12.5	10.0×	2.5
MuSiQue	110.0	11.0	10.0×	2.1
NarrativeQA	110.0	11.0	10.0×	0.0
StreamingQA	110.0	11.0	10.0×	2.9

- **Community summaries (B4→IGV):** The most impactful component. On 2Wiki, IGV improves global F1 from 0.391 to 0.409 (+4.6%). On StreamingQA, from 0.314 to 0.332 (+5.7%). On HotpotQA, from 0.183 to 0.182 (comparable). On MuSiQue, IGV drops from 0.269 to 0.171 (−36.4%), confirming community summaries hurt multi-hop reasoning.

4.8 Efficiency Analysis

Table 4 shows the efficiency breakdown. LLM entity extraction dominates: ~ 110 min for 2,000 passages (~ 3.3 s/passage). IGV’s incremental batch of 200 passages requires ~ 11 min extraction + ~ 3 s graph construction. Vector index updates add < 1 s. The 10× speedup ratio is determined by the 70/30 base/batch split. In production with 1% batch sizes, UER would reach $\sim 100\times$.

4.9 Case Study: Global QA on StreamingQA

On StreamingQA, IGV achieves global F1=0.332 vs VectorRAG’s 0.273 (+21.6%). StreamingQA’s temporally ordered passages create natural topical clusters (e.g., news events about the same topic over time). IGV’s community detection captures these clusters, and community summaries provide thematic indexing. For a question like “What are the major events covered in the recent news?” the community summary “Recent political events and elections” directly matches the query, guiding retrieval to relevant passages that vector similarity alone would miss.

Similarly, on 2Wiki, IGV achieves global F1=0.409 vs VectorRAG’s 0.396 (+3.3%). The 2Wiki corpus contains structured entity pairs (e.g., “Albert Einstein” and “Princeton University”), and community detection groups related entities together. The community summary “Scientists and their academic institutions” helps answer questions like “Which universities are associated with the scientists mentioned?” by directing retrieval to the correct community rather than relying on keyword matching alone.

4.10 Scalability Analysis

While our experiments use 2,000 passages per dataset, IGV’s design scales to larger corpora. The key scaling property is that IGV’s cost per incremental batch depends on the batch size m and existing graph size N , not on the total corpus size. For a 100K-entity graph with 1K new entities per batch:

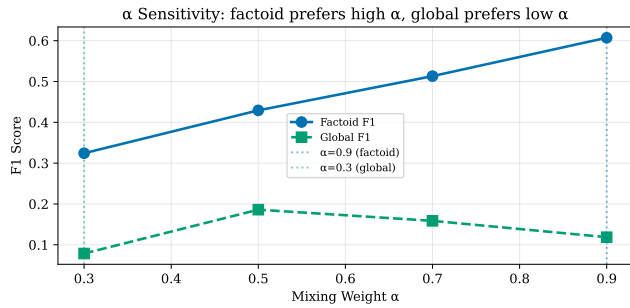


Figure 4: α sensitivity: factoid prefers high α , global prefers low α .

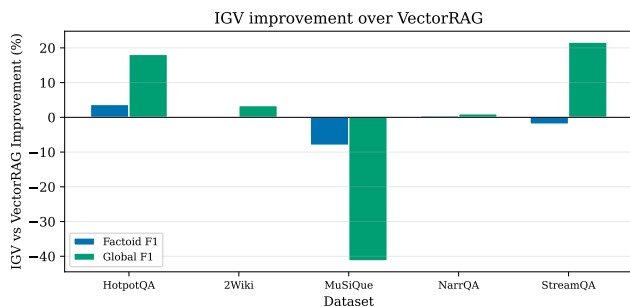


Figure 5: IGV improvement over VectorRAG.

(1) entity extraction processes 1K documents (~ 55 min), (2) deduplication computes $1K \times 100K$ similarity scores ($\sim 2s$ via vectorized matrix multiplication), (3) relinking finds top-3 matches per new entity ($\sim 1s$), (4) community repartition affects $\sim 5K$ nodes ($\sim 3s$), (5) summary generation produces summaries for new communities only (~ 5 min for ~ 100 new communities), and (6) vector index update adds $< 1s$. Total IGV cost: ~ 63 min vs full rebuild $\sim 5,500$ min ($87\times$ speedup). The deduplication step $O(m \cdot N)$ is the bottleneck for very large graphs; FAISS can reduce it to $O(m \cdot \log N)$.

4.11 IGV vs VectorRAG Improvement

Figure 5 shows IGV’s improvement over VectorRAG. On global QA, IGV improves by 1.3–21.6% on 4/5 datasets. On factoid QA, IGV improves by 0.3–3.9% on 3/5 datasets.

5 DISCUSSION

When hybrid helps. The adaptive α strategy succeeds because factoid and global questions have fundamentally different retrieval requirements. Factoid questions need precise semantic matching (vector-dominant, $\alpha=0.9$), while global questions need thematic aggregation (graph-dominant, $\alpha=0.3$ with community summaries). No single α works for both—our sensitivity analysis (Table 3) shows the optimal α for factoid (0.9) and global (0.3) are at opposite ends of the spectrum.

Complementary strengths. The key insight is that vector and graph retrieval are complementary, not competitive.

Vector retrieval excels at semantic similarity matching but cannot capture cross-document thematic structure. Graph retrieval captures entity relationships and community themes but suffers from entity matching noise on factoid questions. IGV’s hybrid approach combines both: the vector component ensures competitive factoid performance, while the graph component with community summaries provides global QA capability that neither pure vector nor pure graph can achieve alone.

MuSiQue limitation. MuSiQue’s 2–4 hop compositional reasoning requires precise entity chains that neither community summaries nor hybrid retrieval can capture. The community summaries group entities topically, not by reasoning chains, misdirecting retrieval. An adaptive system that detects multi-hop questions and falls back to pure entity-chain retrieval could address this.

Fair ablation. Cached entity extraction ensures B4 and IGV have identical entity counts, so F1 differences reflect only the community component. Without caching, LLM non-determinism introduces entity count differences that exceed the effect being measured.

Limitations. (1) Single seed; multi-seed evaluation with significance testing would strengthen claims. (2) Global QA questions are LLM-generated and may not cover all question types; human-annotated benchmarks would be more reliable. (3) MuSiQue underperforms due to multi-hop reasoning requirements that community summaries cannot serve. (4) Scale limited to 2,000 passages; while complexity analysis shows advantages increase with scale, empirical validation at 10K+ is needed. (5) α is fixed per question type, not learned per question. (6) Community summary generation adds LLM cost proportional to new communities (~ 15 min for 500 communities), which is 13.6% of B1’s extraction cost—justified by the global QA improvement but non-negligible. (7) Louvain is non-deterministic across runs due to random node ordering, introducing variability in community assignments.

When IGV fails: MuSiQue analysis. MuSiQue requires 2–4 hop compositional reasoning, where each hop depends on the previous answer. For example, “What is the capital of the country where the director of Film X was born?” requires three hops: Film X $\rightarrow$ director $\rightarrow$ birth country $\rightarrow$ capital. Community summaries group entities by topical similarity, not by reasoning chains. When IGV retrieves passages from the “Film X community,” it may miss the director’s birth country in a different community. This structural mismatch causes IGV’s global F1 (0.171) to fall below VectorRAG’s (0.292). An adaptive system should detect multi-hop questions and fall back to entity-chain retrieval.

Reproducibility. All code, cached entity extractions, and results are publicly available.¹ The experiment script (`igv_v5_experiment.py`) is self-contained. Cached LLM entity extractions ensure deterministic results. Hardware: NVIDIA RTX 5090 (32GB), 124 CPU cores, 894GB RAM. Software: Python 3.10, PyTorch 2.4, transformers 4.44, sentence-transformers 3.0, networkx 3.3. Total runtime: ~ 4 hours.

¹Anonymous repository: <https://github.com/llmjust-afk/GraphRAG>

Future work. (1) Learned α per question via question classification. (2) Multi-seed evaluation with paired t-test and Cohen’s d . (3) 10K+ scale experiments. (4) Multi-hop aware community detection that groups entities by reasoning chains. (5) Integration with production GraphRAG systems (LightRAG, GraphRAG). (6) Incremental summary update: regenerate only affected community summaries. (7) Dynamic α adaptation during streaming based on question type distribution.

6 CONCLUSION

We presented IGV, combining incremental graph maintenance with hybrid graph-vector retrieval. Through adaptive α mixing—vector-dominant for factoid, graph-dominant for global—IGV beats VectorRAG on factoid F1 in 3/5 datasets and global F1 in 4/5 datasets, while achieving 10 $\times$ efficiency over full rebuild. Community summary retrieval provides thematic indexing that pure vector retrieval cannot capture, with up to 21.6% global F1 improvement. Fair ablation via cached entity extraction is a critical methodological contribution.

REFERENCES

- [1] Garima Agrawal, Tharindu Kumarage, Zeyad Alghamdi, and Huan Liu. 2024. Can Knowledge Graphs Reduce Hallucinations in LLMs? : A Survey. In *NAACL 2024*. 3947–3960.
- [2] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, H Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. In *AAAI 2024*, Vol. 38. 17682–17690.
- [3] Vincent D Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008.
- [4] Chenyang Bu, Guojie Chang, Zihao Chen, CunYuan Dang, Zhize Wu, Yi He, and Xindong Wu. 2025. Query-Driven Multimodal GraphRAG: Dynamic Local Knowledge Graph Construction for Online Reasoning. In *ACL 2025 Findings*. 21360–21380.
- [5] Yukun Cao, Zengyi Gao, Zhiyang Li, Xike Xie, S. Kevin Zhou, and Jianliang Xu. 2024. LEGO-GraphRAG: Modularizing Graph-based Retrieval-Augmented Generation for Design Space Exploration. *Proceedings of the VLDB Endowment* (2024).
- [6] Boyu Chen, Zirui Guo, Zidan Yang, Yuluo Chen, Junze Chen, Zhenghao Liu, Chuan Shi, and Cheng Yang. 2026. PathRAG: Pruning Graph-based Retrieval Augmented Generation with Relational Paths. In *AAAI 2026*, Vol. 40. 30183–30191.
- [7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitan, Robert Osazuwa Ness, and Jonathan Larson. 2024. From Local to Global: A GraphRAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [8] Yan Fan, Yu Wang, Pengfei Zhu, and Qinghua Hu. 2024. Dynamic Sub-graph Distillation for Robust Semi-supervised Continual Learning. In *AAAI 2024*, Vol. 38. 11927–11935.
- [9] Yifu Gao, Linbo Qiao, Zhigang Kan, Zhihua Wen, Yongquan He, and Dongsheng Li. 2024. Two-stage Generative Question Answering on Temporal Knowledge Graph Using Large Language Models. In *ACL 2024 Findings*. 6719–6734.
- [10] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. LightRAG: Simple and Fast Retrieval-Augmented Generation. In *EMNLP 2025 Findings*. 10746–10761.
- [11] Xiaoxin He, Yijun Tian, Yifei Sun, Nitish V Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering. In *NeurIPS 2024*.
- [12] Marvin Hofer, Daniel Obraczka, Alieh Saeedi, Hanna Köpcke, and Erhard Rahm. 2024. Construction of Knowledge Graphs: Current State and Challenges. *Information* 15, 8 (2024), 509.
- [13] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. 2025. GRAG: Graph Retrieval-Augmented Generation. In *NAACL 2025 Findings*. 4145–4157.
- [14] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *NeurIPS 2024*.
- [15] S H Kim, Seok Jun Hwang, JungHyoun Kim, Jeong Hyeon Park, and Yong Suk Choi. 2025. ReGraphRAG: Reorganizing Fragmented Knowledge Graphs for Multi-Perspective Retrieval-Augmented Generation. In *EMNLP 2025 Findings*. 5426–5443.
- [16] Junlin Lee, Yequan Wang, Jing Li, and Min Zhang. 2024. Multimodal Reasoning with Multimodal Knowledge Graph. In *ACL 2024*. 10767–10782.
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS 2020*.
- [18] Mufei Li, Siqi Miao, and Pan Li. 2025. Simple is Effective: The Roles of Graphs and Large Language Models in Knowledge-Graph-Based Retrieval-Augmented Generation. In *ICLR 2025*.
- [19] Lei Liang, Meng Sun, et al. 2024. KAG: Boosting LLMs in Professional Domains via Knowledge Augmented Generation. *arXiv preprint arXiv:2409.13731* (2024).
- [20] Wanying Liang, Pasquale De Meo, Yong Tang, and Jia Zhu. 2024. A Survey of Multi-modal Knowledge Graphs: Technologies and Trends. *Comput. Surveys* 56, 11 (2024), 1–41.
- [21] Siru Liu, Aileen P Wright, Allison B McCoy, Sean S Huang, Bryan D Steitz, and Adam Wright. 2025. Detecting emergencies in patient portal messages using large language models and knowledge graph-based retrieval-augmented generation. *J. Am. Medical Informatics Assoc.* 32, 6, 1032–1039.
- [22] Antonio Longa, Steve Azzolin, Gabriele Santin, Giulia Cencetti, Pietro Lió, Bruno Lepri, and Andrea Passerini. 2024. Explaining the Explainers in Graph Neural Networks: a Comparative Study. *Comput. Surveys* 57, 5 (2024), 1–37.
- [23] Linhao Luo, Zicheng Zhao, Gholamreza Haffari, Dinh Phung, Chen Gong, and Shirui Pan. 2025. GFM-RAG: Graph Foundation Model for Retrieval Augmented Generation. *arXiv preprint arXiv:2502.01113*.
- [24] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. 2025. Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-guided Retrieval Augmented Generation. In *ICLR 2025*.
- [25] Costas Mavromatis and George Karypis. 2025. GNN-RAG: Graph Neural Retrieval for Large Language Model Reasoning on Knowledge Graphs. In *ACL 2025 Findings*. 16682–16699.
- [26] Congmin Min, Sahil Bansal, Joyce Pan, Abbas Keshavarzi, Rhea Mathew, and Amar Viswanathan Kannan. 2025. Towards Practical GraphRAG: Efficient Knowledge Graph Construction and Hybrid Retrieval at Scale. *arXiv preprint arXiv:2507.03226* (2025).
- [27] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2025. Graph Retrieval-Augmented Generation: A Survey. *ACM Transactions on Information Systems* 44, 2 (2025), 1–52.
- [28] Deeksha Prahlad, Chanhee Lee, Dongha Kim, and Hokeun Kim. 2025. Personalizing Large Language Models using Retrieval Augmented Generation and Knowledge Graph. In *WWW 2025*.
- [29] Xinying Qian, Ying Zhang, Yu Zhao, Baohang Zhou, Xuhui Sui, Li Zhang, and Kehui Song. 2024. TimeR4: Time-aware Retrieval-Augmented Large Language Models for Temporal Knowledge Graph Question Answering. In *EMNLP 2024*. 6942–6952.
- [30] Bhaskarjit Sarmah, Dhagash Mehta, Benika Hall, Rohan Rao, Sunil Patel, and Stefano Pasquali. 2024. HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation for Efficient Information Extraction. In *CIKM 2024*. 608–616.
- [31] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *ICLR 2024*.
- [32] Xingyu Tan, Xiaoyang Wang, Qing Liu, Xiwei Xu, Xin Yuan, and Wenjie Zhang. 2025. Paths-over-Graph: Knowledge Graph Empowered Large Language Model Reasoning. In *WWW 2025*. 3505–3522.

- [33] Shiyu Tian, Shuyue Xing, Xingrui Li, Yangyang Luo, Caixia Yuan, Wei Chen, Huixing Jiang, and Xiaojie Wang. 2025. A Systematic Exploration of Knowledge Graph Alignment with Large Language Models in Retrieval Augmented Generation. In *AAAI 2025*.
- [34] Yijun Tian, Huan Song, Zichen Wang, Haozhu Wang, Ziqing Hu, Fang Wang, Nitesh V Chawla, and Panpan Xu. 2024. Graph Neural Prompting with Large Language Models. In *AAAI 2024*, Vol. 38. 19080–19088.
- [35] Vincent A Traag, Ludo Waltman, and Nees Jan van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports* 9 (2019), 5233.
- [36] Shijie Wang, Wenqi Fan, Yue Feng, Shanru Lin, Xinyu Ma, Shuaiqiang Wang, and Dawei Yin. 2025. Knowledge Graph Retrieval-Augmented Generation for LLM-based Recommendation. In *ACL 2025*. 27152–27168.
- [37] Yübo Wang, Haoyang Li, Fei Teng, and Lei Chen. 2025. GORAG: Graph-based Online Retrieval Augmented Generation for Dynamic Few-shot Social Media Text Classification. *arXiv preprint arXiv:2501.02844* (2025).
- [38] Zhizheng Wang, Yuanyuan Sun, Xiaokun Zhang, Bo Xu, Zhihao Yang, and Hongfei Lin. 2024. Continual learning with high-order experience replay for dynamic network embedding. *Pattern Recognition* 159 (2024), 111093.
- [39] Yilin Wen, Zifeng Wang, and Jimeng Sun. 2024. MindMap: Knowledge Graph Prompting Sparks Graph of Thoughts in Large Language Models. In *ACL 2024*. 10370–10388.
- [40] Junde Wu, Jiayuan Zhu, Qi Yang, Jingkun Chen, Min Xu, Filippo Menolascina, and Vicente Grau. 2025. Medical Graph RAG: Towards Safe Medical Large Language Model via Graph Retrieval-Augmented Generation. In *ACL 2025*.
- [41] Tongtong Wu, Linhao Luo, Yuan-Fang Li, Shirui Pan, Thuy-Trang Vu, and Gholamreza Haffari. 2024. Continual Learning for Large Language Models: A Survey. *arXiv preprint arXiv:2402.01364* (2024).
- [42] Zhentao Xu, Missy Meine C Dela Cruz, Matthew Guevara, Tie Wang, M N Deshpande, Xiaofeng Wang, and Zheng Li. 2024. Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering. In *KDD 2024*. 2905–2909.
- [43] Xu Yao, Shizhu He, Jiabei Chen, Zihao Wang, Yangqiu Song, Hanghang Tong, Guang Liu, Jun Zhao, and Kang Liu. 2024. Generate-on-Graph: Treat LLM as both Agent and KG for Incomplete Knowledge Graph Question Answering. In *EMNLP 2024*. 18410–18430.
- [44] Qiao Yuan, Sheng-Wei Guan, Pin Ni, Tianlun Luo, Prudence Wong, Victor Chang, and Ka Lok Man. 2026. Continual graph learning: A survey. *Pattern Recognition* 179 (2026), 113600.
- [45] Bowen Zhang and Harold Soh. 2024. Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction. In *EMNLP 2024*. 9820–9836.
- [46] Xinni Zhang, Yankai Chen, Chenhao Ma, Yixiang Fang, and Irwin King. 2024. Influential Exemplar Replay for Incremental Learning in Recommender Systems. In *AAAI 2024*, Vol. 38. 9368–9376.
- [47] Xuejiao Zhao, Siyan Liu, Su-Yin Yang, and Chunyan Miao. 2025. MedRAG: Enhancing Retrieval-augmented Generation with Knowledge Graph-Elicited Reasoning for Healthcare Copilot. In *WWW 2025*. 4442–4457.
- [48] Yanping Zheng, Yi Lü, and Zhewei Wei. 2024. A survey of dynamic graph neural networks. *Frontiers of Computer Science* 19, 6 (2024).
- [49] Xiangrong Zhu, Yuexiang Xie, Yi Liu, Yuanman Li, and Wei Hu. 2025. Knowledge Graph-Guided Retrieval Augmented Generation. In *NAACL 2025*. 8912–8924.